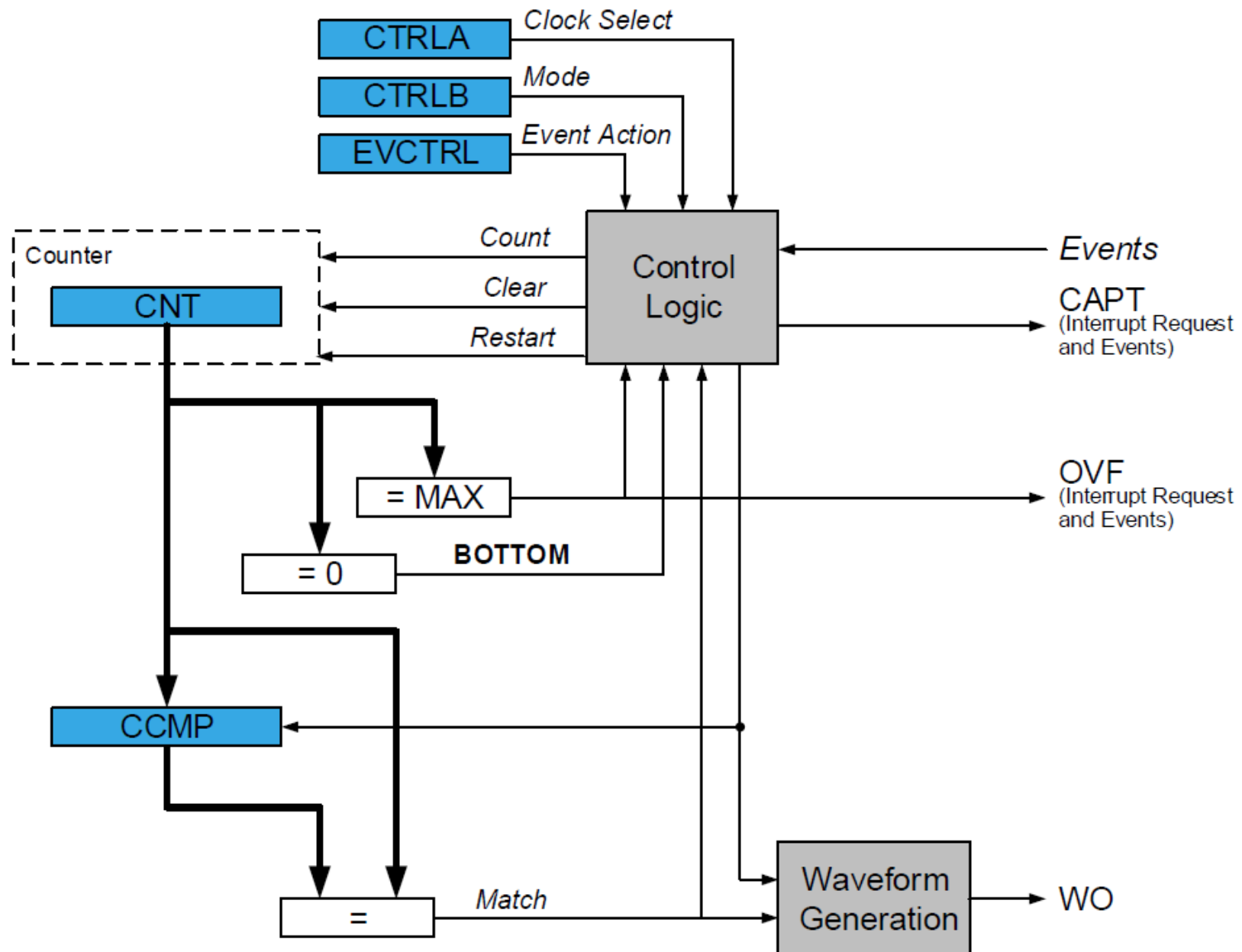


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 9 - TCB Timers (Periodic Interrupts and One-Shot Timing)



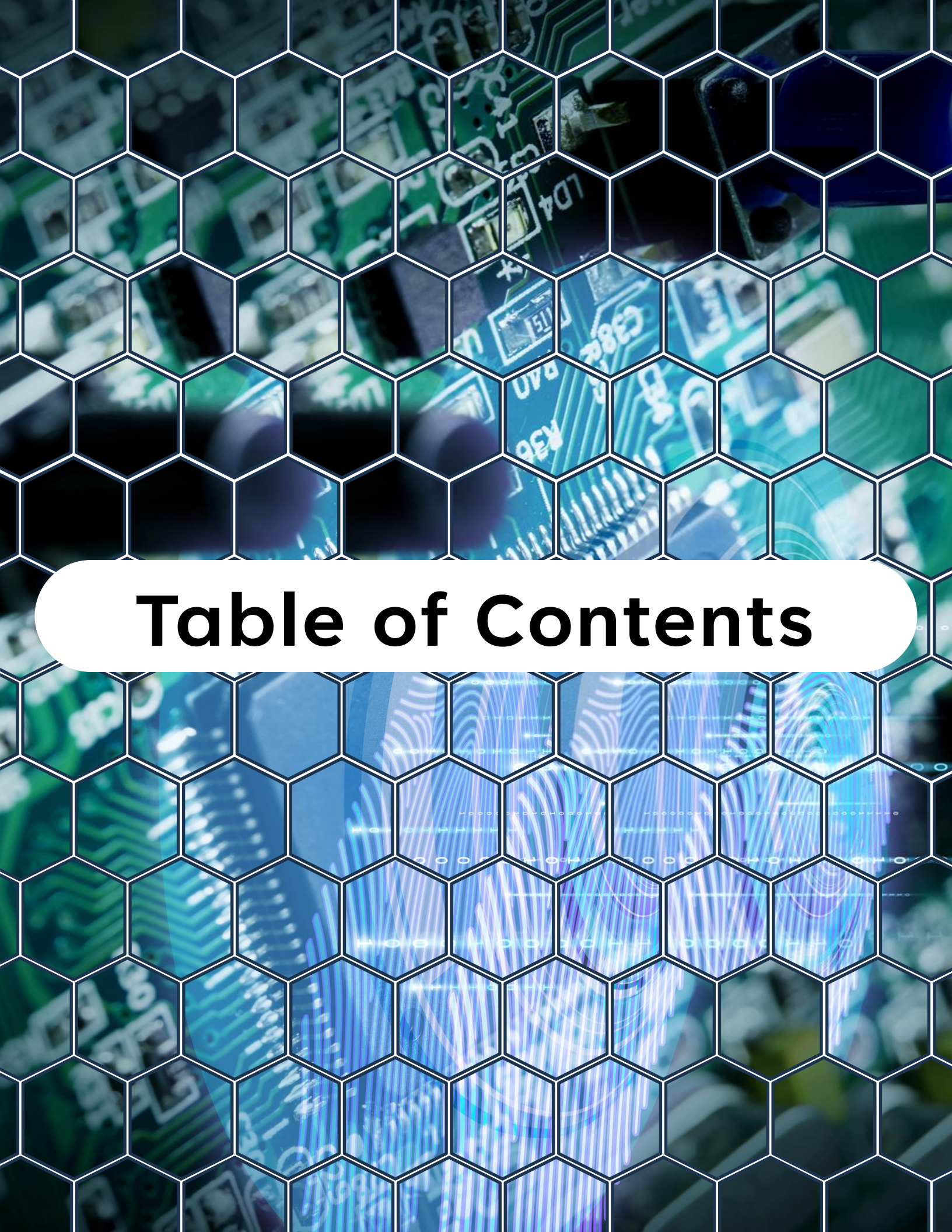
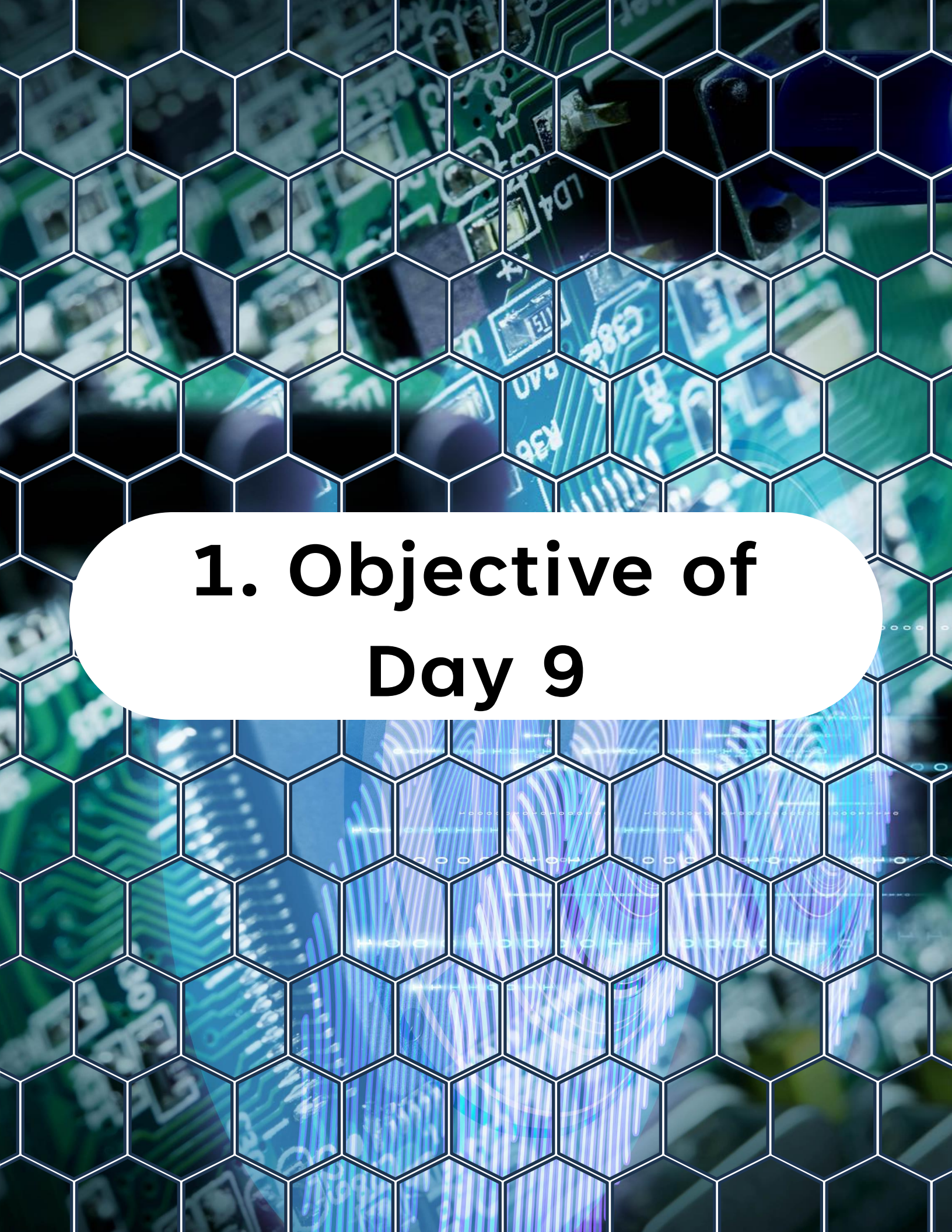


Table of Contents

Table of Contents

1. Objective of Day 9
2. Why TCB Matters
3. TCB Overview (What It Is)
4. Clocking: Where TCB Gets Its Time
5. Two Patterns You Will Use Constantly
6. Timer Math (Periodic or One-Shot)
7. Periodic Interrupt Example (1 ms Tick)
8. Using the Tick (No Delay Loops)
9. One-Shot Timeout Example
10. Making One-Shots Practical (Prescaler or Tick-Based)
11. Debugging TCB (The Usual Failures)
12. What You Learned Today
13. What Comes Next



1. Objective of Day 9

1. Objective of Day 9

Today you will master **TCB**, the lightweight workhorse timer in AVR DA devices.

TCB is what you reach for when you want:

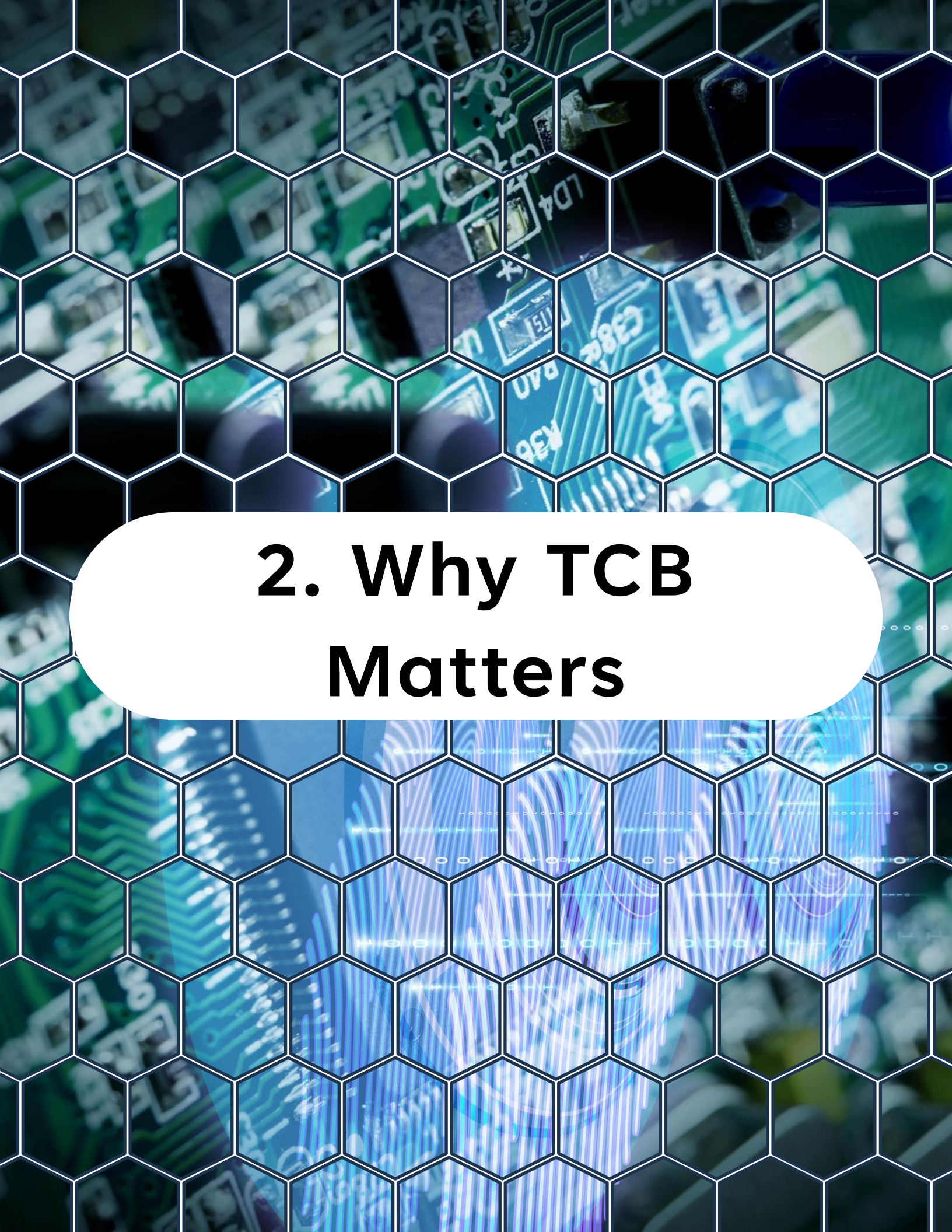
- A clean periodic interrupt (system tick)
- A one-shot timeout (watchdog-like supervision, comms timeout, debounce window)
- Minimal configuration and overhead

By the end of this day, you will:

- Understand what TCB is and how it differs from TCA/RTC
- Configure TCB for periodic interrupts
- Configure TCB for one-shot timing

1. Objective of Day 9

- Build a tiny timebase for later days
(debounce, scheduling, timeouts)
- Learn the common mistakes that cause "ISR
never fires" problems



2. Why TCB Matters

2. Why TCB Matters

TCA is powerful but can feel heavy when you just need a timer interrupt.

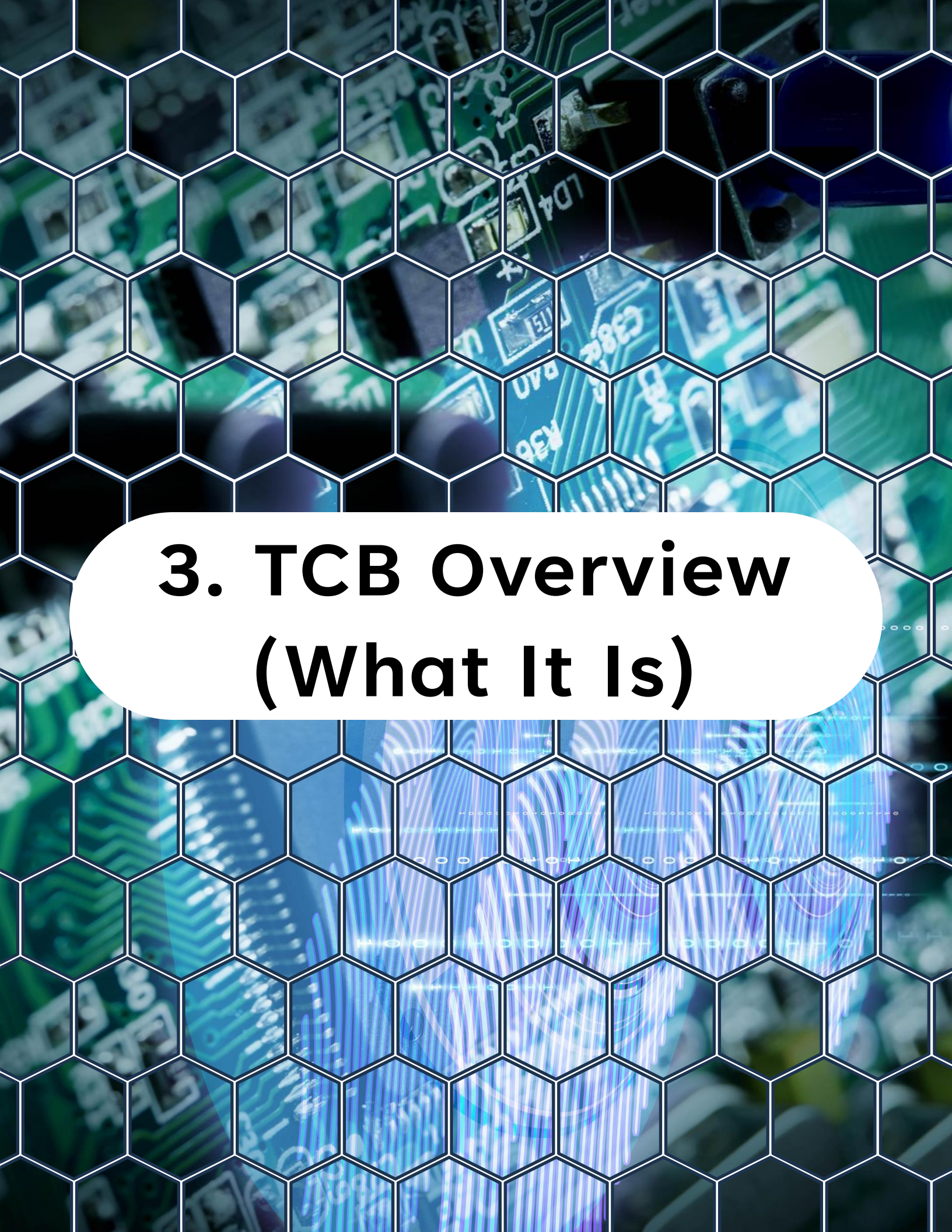
TCB is simple:

- One channel
- One compare value (**CCMP**)
- Multiple counting modes
- Very predictable behavior

In real projects, you often use:

- **TCA** for PWM generation
- **TCB** for scheduling and timeouts
- **RTC** for long time / low power

This division keeps designs clean.



3. TCB Overview (What It Is)

3. TCB Overview (What It Is)

Each TCB peripheral is a 16-bit timer with:

- Counter register (**CNT**)
- Compare/Capture register (**CCMP**)
- Control registers (**CTRLA/CTRLB**)
- Interrupt flags and enable bits

Key behavior:

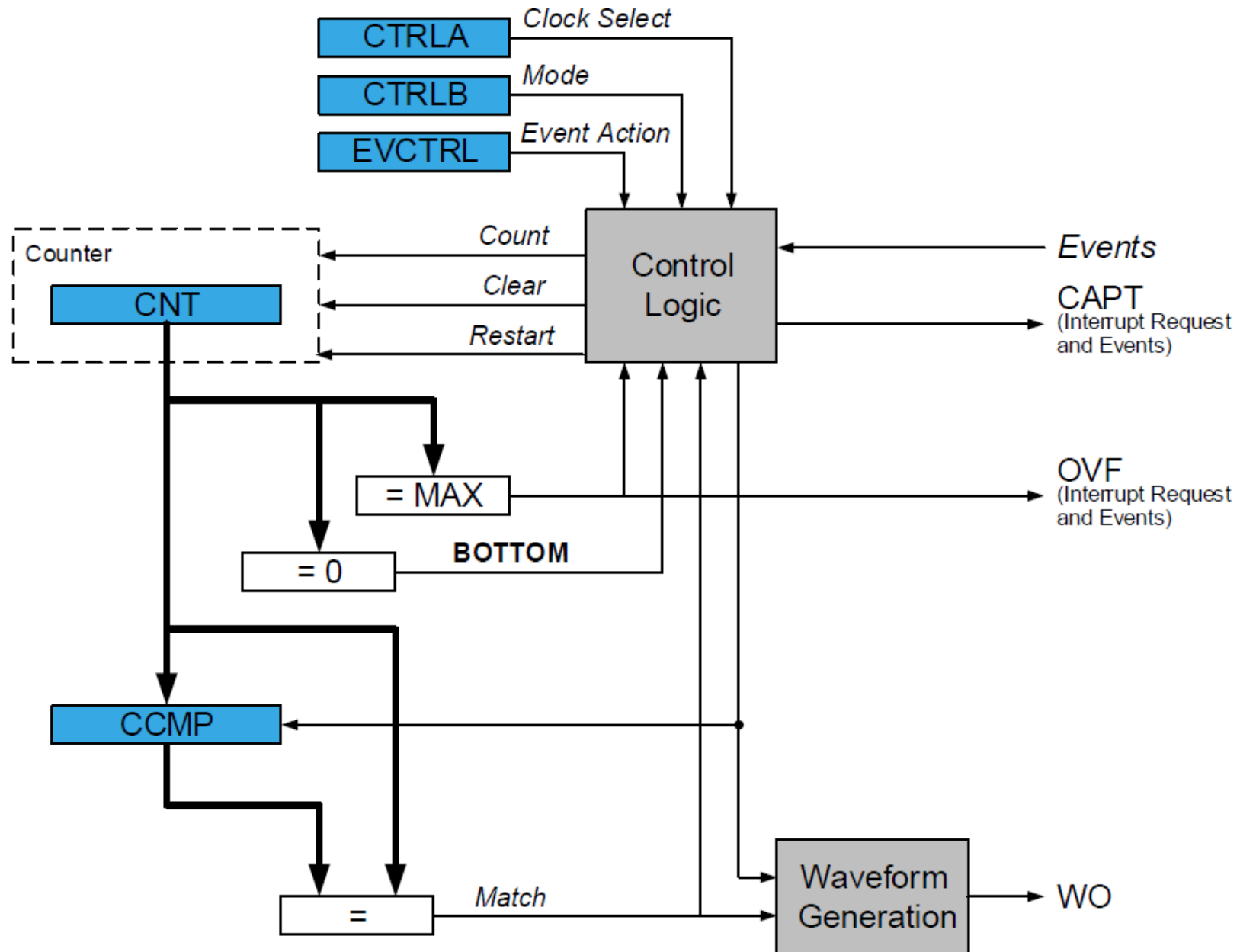
It counts clock ticks (usually **CLK_PER**)

When it reaches **CCMP** (or certain events happen), it triggers an interrupt

TCB is great for precise time intervals.

3. TCB Overview (What It Is)

Timer/Counter Type B Block Diagram





4. Clocking: Where TCB Gets Its Time

4. Clocking: Where TCB Gets Its Time

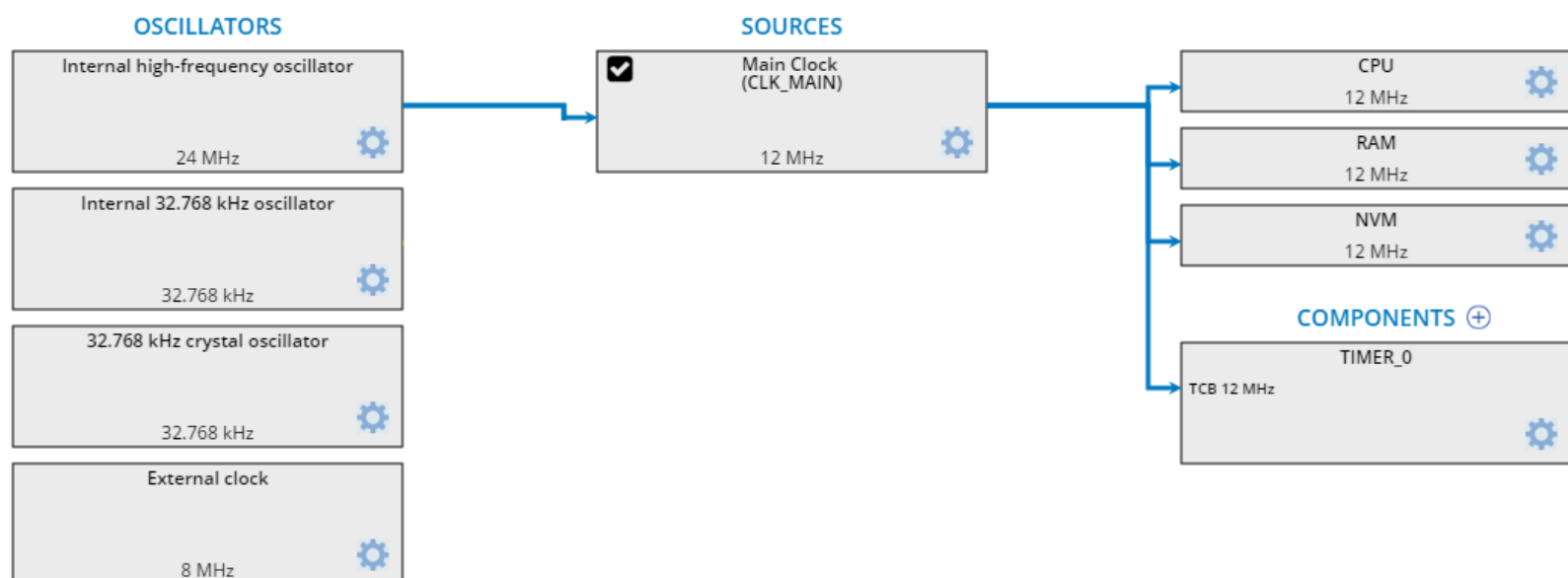
TCB typically runs from **CLK_PER** (the peripheral clock), which is usually derived from the main clock.

So, your timing depends on:

- System clock configuration (Day 3)
- Any clock prescalers
- The TCB clock select setting

If your clock is wrong, your timer math is wrong.

TCB0 Clock Source





5. Two Patterns You Will Use Constantly

5. Two Patterns You Will Use Constantly

Pattern A: Periodic Interrupt ("Tick")

- Timer runs continuously
- Interrupt triggers at a fixed interval
- ISR runs repeatedly

Use cases:

- Blinking LED without delays
- Cooperative schedulers
- Periodic sensor sampling
- Timekeeping in milliseconds

Pattern B: One-Shot Timeout

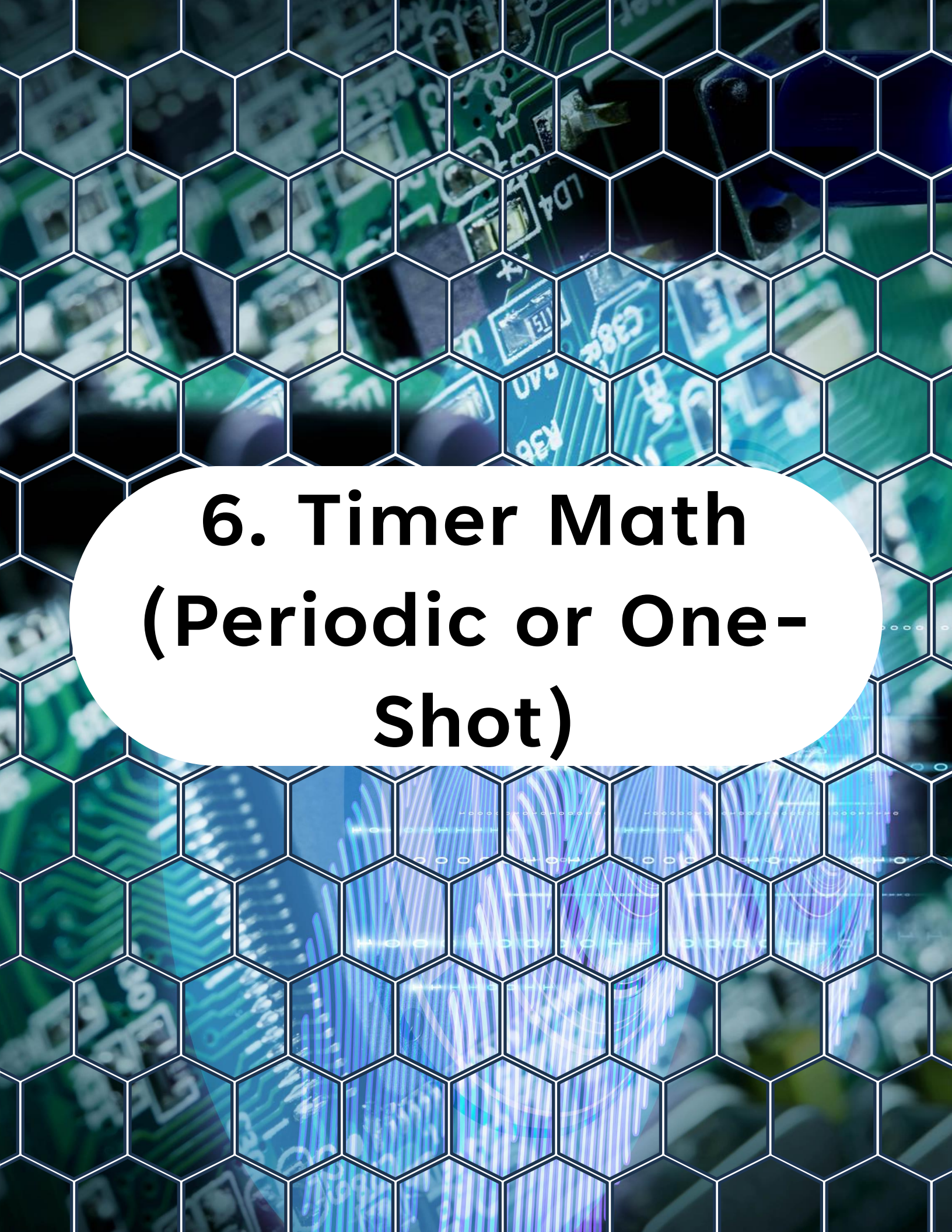
- Timer starts once
- Interrupt fires once after the delay
- Timer stops (or you stop it)

5. Two Patterns You Will Use Constantly

- Sets a flag, wakes a task, triggers recovery

Use cases:

- Communication timeout
- Debounce window
- Startup delay
- Fault handling "grace period"



6. Timer Math (Periodic or One-Shot)

6. Timer Math (Periodic or One-Shot)

Basic idea:

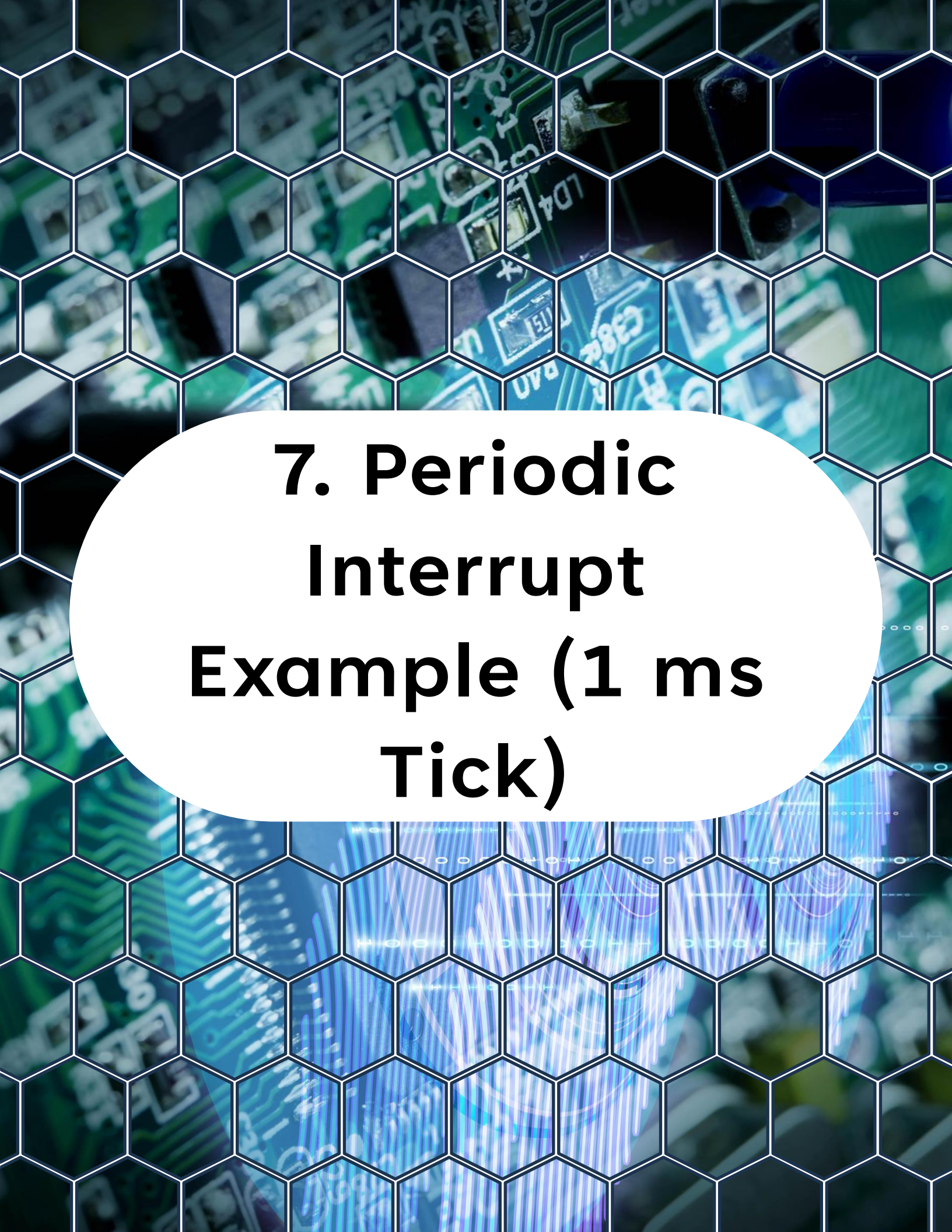
- **TCB** ticks at F_{tcb}
- You load **CCMP** with number of ticks until event

$$\text{Ticks} = F_{\text{tcb}} * \text{interval_seconds}$$

Example (1 ms tick):

- $F_{\text{tcb}} = 1,000,000 \text{ Hz}$
- $\text{interval} = 0.001 \text{ s}$
- $\text{ticks} = 1000$

In most designs, you pick a prescaler or clock so the math is simple.



7. Periodic Interrupt Example (1 ms Tick)

7. Periodic Interrupt Example (1 ms Tick)

This example builds a classic 1 ms system tick using TCB0.

Assumptions:

- $F_{CPU} = 24 \text{ MHz}$
- $CLK_PER = 24 \text{ MHz}$
- We will use a prescaler inside TCB by selecting a clock division (if available) or simply load CCMP accordingly

For learning, we will keep it direct:

- Use CLK_PER (no extra prescaler here)
- Set CCMP so interrupt fires every 1 ms

1 ms at 24 MHz = 24000 ticks

7. Periodic Interrupt Example (1 ms Tick)

Code: TCB0 init for periodic 1 ms interrupt



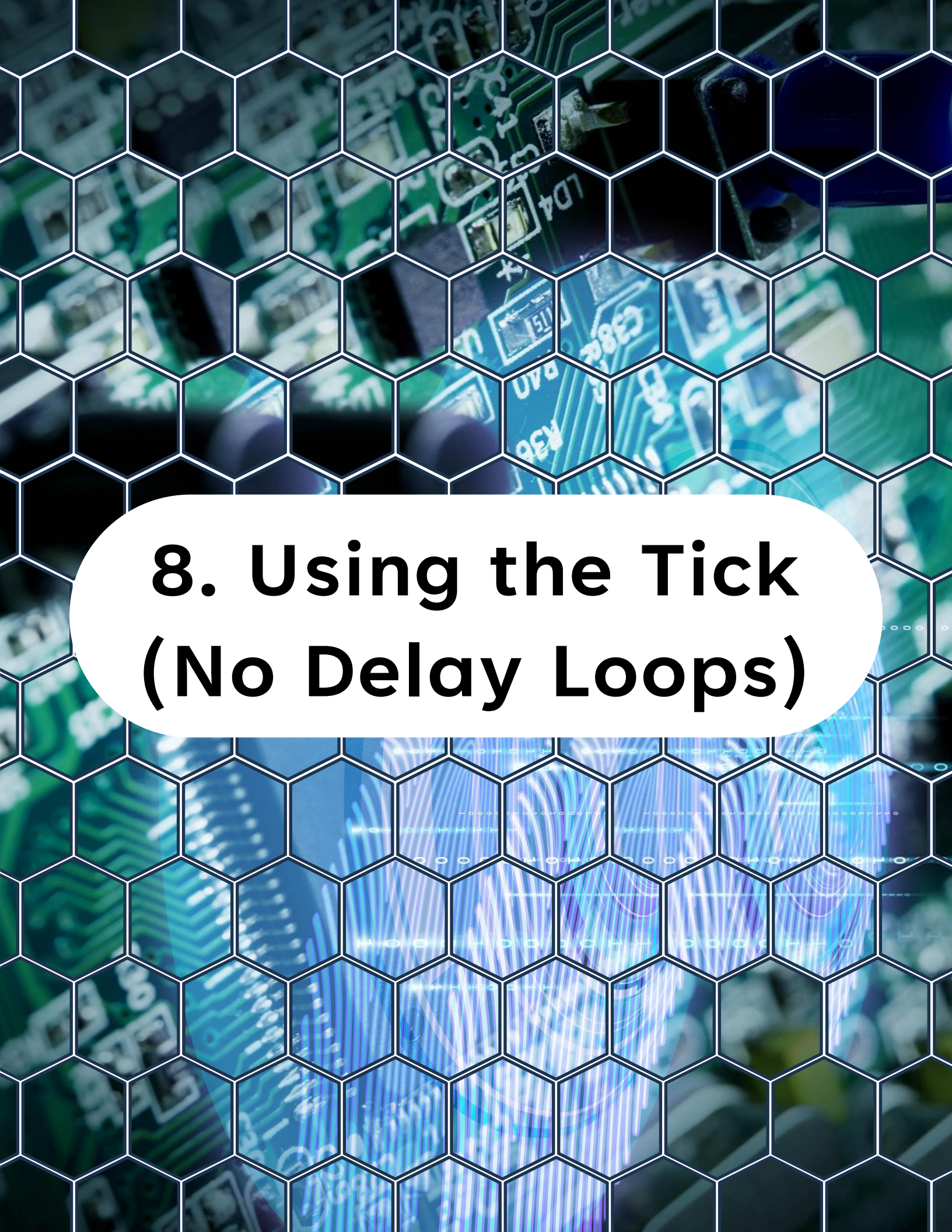
```
1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3
4  #define F_CPU_HZ          (24000000UL)
5  #define TICK_HZ           (1000UL)
6  #define TCB_TICKS        (F_CPU_HZ / TICK_HZ)
7
8  static volatile uint32_t g_ms_ticks = 0;
9
10 static void tcb0_tick_init(void)
11 {
12     /* Disable TCB0 before configuration */
13     TCB0.CTRLA = 0;
14
15     /* Periodic interrupt mode */
16     TCB0.CTRLB = TCB_CNTMODE_INT_gc;
17
18     /* Set compare value for 1 ms interval */
19     TCB0.CCMP = (uint16_t)(TCB_TICKS - 1U);
20
21     /* Clear any pending interrupt flags */
22     TCB0.INTFLAGS = TCB_CAPT_bm;
23
24     /* Enable capture/compare interrupt */
25     TCB0.INTCTRL = TCB_CAPT_bm;
26
```

7. Periodic Interrupt Example (1 ms Tick)

```
27     /* Enable timer, use CLK_PER */
28     TCB0.CTRLA = TCB_ENABLE_bm;
29 }
30
31 ISR(TCB0_INT_vect)
32 {
33     /* Clear interrupt flag */
34     TCB0.INTFLAGS = TCB_CAPT_bm;
35
36     /* Increment millisecond counter */
37     g_ms_ticks++;
38 }
```

Notes:

- The ISR is short and deterministic
- This becomes your timing backbone for the next weeks
- Avoid heavy work inside the ISR



8. Using the Tick (No Delay Loops)

8. Using the Tick (No Delay Loops)

Now you can blink an LED without delay loops.

Code: blink based on tick counter

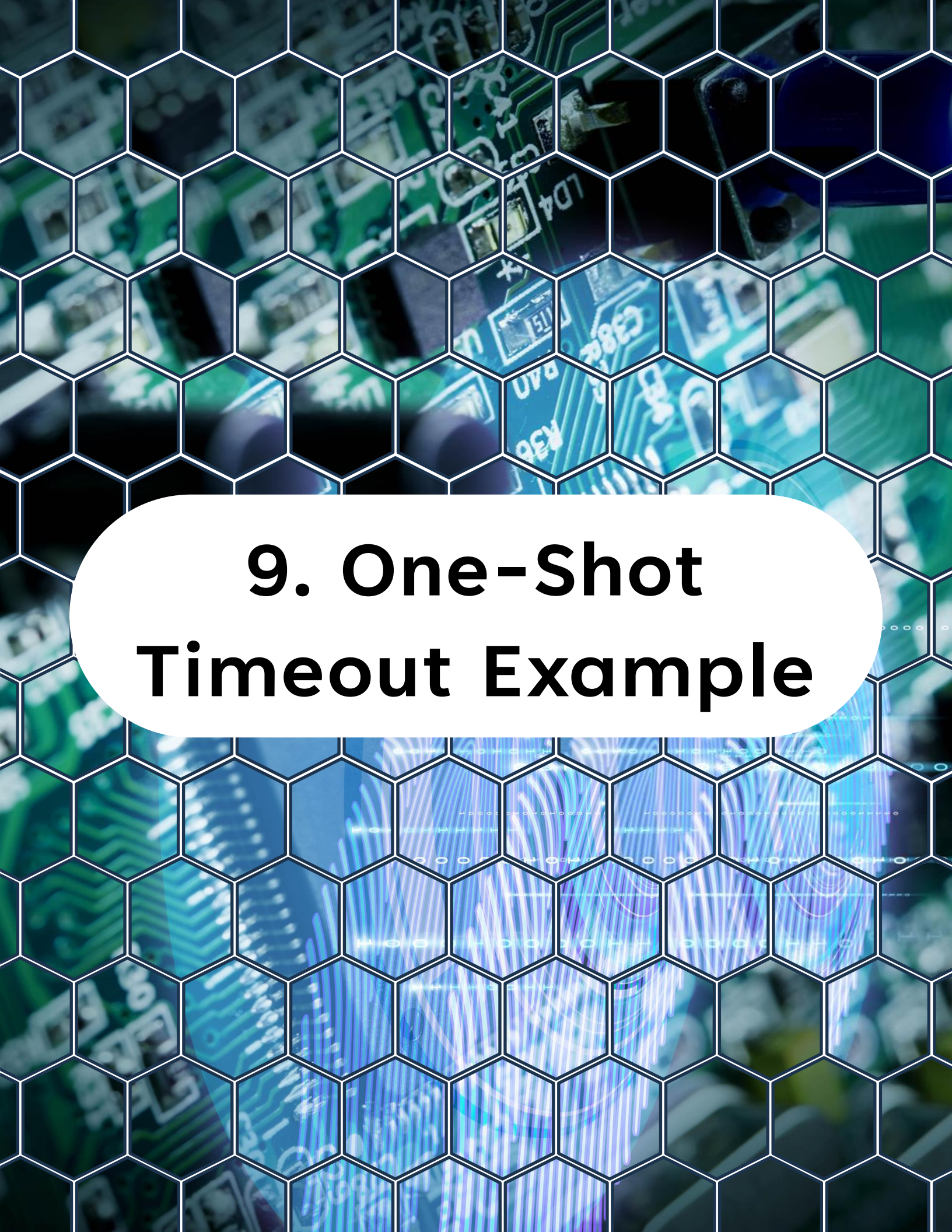
```
1 static void gpio_led_init(void)
2 {
3     /* Example LED on PA0 (adjust to your real LED pin) */
4     PORTA.DIRSET = PIN0_bm;
5     PORTA.OUTCLR = PIN0_bm;
6 }
7
8 static void gpio_led_toggle(void)
9 {
10     PORTA.OUTTGL = PIN0_bm;
11 }
12
13 int main(void)
14 {
15     gpio_led_init();
16     tcb0_tick_init();
17     sei();
18
19     uint32_t last = 0;
20
21     while (1)
22     {
23         if ((g_ms_ticks - last) >= 500U)
24         {
25             last = g_ms_ticks;
```

8. Using the Tick (No Delay Loops)

```
20
21     while (1)
22     {
23         if ((g_ms_ticks - last) >= 500U)
24         {
25             last = g_ms_ticks;
26             gpio_led_toggle();
27         }
28     }
29 }
```

This is the correct embedded pattern:

- main loop does work when time says so
- ISR provides timebase only
- no blocking delays

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', 'U7A', and 'K38'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb-like texture. In the center, a white rounded rectangle contains the title text.

9. One-Shot Timeout Example

9. One-Shot Timeout Example

Now you will use **TCB** as a one-shot timer that fires once after a delay.

There are multiple ways to implement "one-shot" depending on mode support and preference. The cleanest learning approach:

- Configure TCB in periodic interrupt mode
- Enable it
- In ISR: disable TCB so it does not repeat

This is easy to reason about and works well for many use cases.

9. One-Shot Timeout Example

Code: one-shot timer using TCB1



```
1 static volatile uint8_t g_oneshot_fired = 0;
2
3 static void tcb1_oneshot_start_us(uint16_t delay_us)
4 {
5     /*
6      * delay_us is in microseconds.
7      * If CLK_PER is 24 MHz, then ticks per us = 24.
8      * ticks = delay_us * 24
9      */
10    uint32_t ticks = (uint32_t)delay_us * (F_CPU_HZ / 1000000UL);
11
12    if (ticks == 0)
13    {
14        ticks = 1;
15    }
16
17    if (ticks > 0xFFFFUL)
18    {
19        ticks = 0xFFFFUL; /* clamp */
20    }
21
22    g_oneshot_fired = 0;
23
24    TCB1.CTRLA = 0;
25    TCB1.CTRLB = TCB_CNTMODE_INT_gc;
26    TCB1.CCMP = (uint16_t)(ticks - 1U);
27    TCB1.INTFLAGS = TCB_CAPT_bm;
28    TCB1.INTCTRL = TCB_CAPT_bm;
29
30    /* Enable and start */
31    TCB1.CTRLA = TCB_ENABLE_bm;
```

9. One-Shot Timeout Example

```
29
30     /* Enable and start */
31     TCB1.CTRLA = TCB_ENABLE_bm;
32 }
33
34 ISR(TCB1_INT_vect)
35 {
36     /* Clear flag */
37     TCB1.INTFLAGS = TCB_CAPT_bm;
38
39     /* Stop timer to make it one-shot */
40     TCB1.CTRLA &= ~TCB_ENABLE_bm;
41
42     g_oneshot_fired = 1;
43 }
```

Using the one-shot

```
1  int main(void)
2  {
3      gpio_led_init();
4      tcb0_tick_init();
5      sei();
6
7      /* Start a 200000 us one-shot (200 ms) */
8      tcb1_oneshot_start_us(200000U);
9
10     while (1)
11     {
12         if (g_oneshot_fired)
13         {
14             g_oneshot_fired = 0;
15             gpio_led_toggle();
16         }
17     }
18 }
```


9. One-Shot Timeout Example

```
9
10 while (1)
11 {
12     if (g_oneshot_fired)
13     {
14         g_oneshot_fired = 0;
15         gpio_led_toggle();
16     }
17 }
18 }
```

Important reality check:

- A 200 ms delay at 24 MHz requires 4,800,000 ticks, which does not fit in 16 bits.
- This is why prescalers (or chaining ticks) matter.

That limitation is part of the lesson.



10. Making One-Shots Practical (Prescaler or Tick-Based)

10. Making One-Shots Practical (Prescaler or Tick-Based)

Because TCB is 16-bit, long delays need:

- A slower clock (prescale), or
- A higher-level counter using the 1 ms tick

Practical pattern:

one-shot in milliseconds using g_ms_ticks

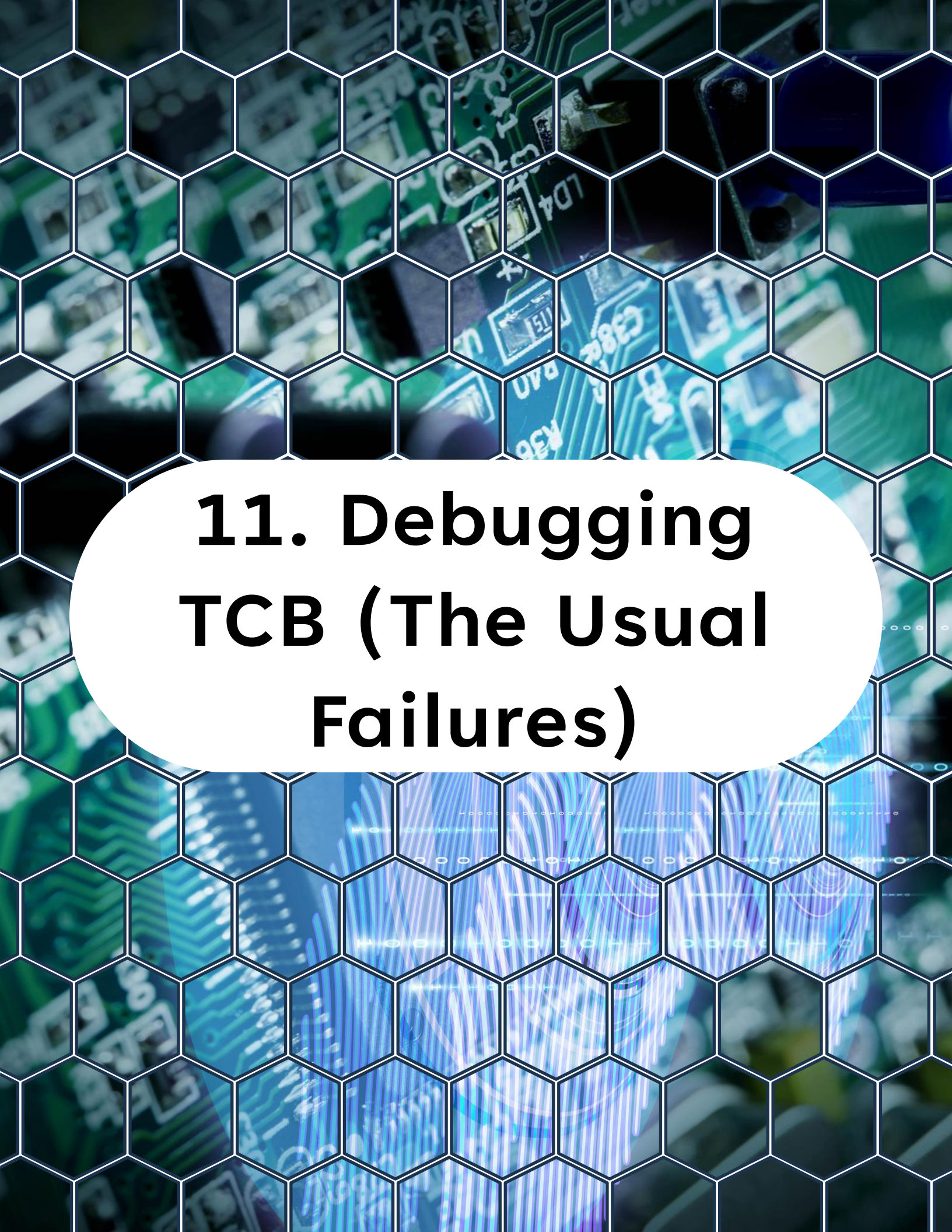
```
1 static uint32_t g_deadline_ms = 0;
2
3 static void oneshot_ms_start(uint32_t delay_ms)
4 {
5     g_deadline_ms = g_ms_ticks + delay_ms;
6 }
7
8 static uint8_t oneshot_ms_expired(void)
9 {
10     return (uint8_t)((int32_t)(g_ms_ticks - g_deadline_ms) >= 0);
11 }
```


10. Making One-Shots Practical (Prescaler or Tick-Based)

This is often the best approach:

- Hardware tick provides timebase
- Software handles long intervals safely

You will use this pattern constantly in real projects.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the title text in bold black font.

11. Debugging TCB (The Usual Failures)

11. Debugging TCB (The Usual Failures)

If the ISR never fires, check these in order:

- Global interrupts enabled? (`sei()`)
- Correct vector name? (`ISR(TCB0_INT_vect)`)
- Interrupt enabled? (`TCB0.INTCTRL`)
- Interrupt flag cleared? (`TCB0.INTFLAGS`)
- Timer enabled? (`TCB0.CTRLA` includes `ENABLE`)
- Clock running? (Day 3 clock sanity)
- Wrong TCB instance? (TCB0 vs TCB1 etc.)

When in doubt:

- Toggle a GPIO pin inside the ISR
- Measure with a scope

This removes guessing.



12. What You Learned Today

12. What You Learned Today

By the end of Day 9, you should be comfortable with:

- The role of TCB in AVR DA designs
- Building a stable periodic interrupt tick
- Using the tick to schedule work without delays
- Implementing a one-shot timeout and understanding its limits
- Debugging timer interrupt issues quickly

You now have a real timing foundation.



13. What Comes Next

13. What Comes Next

Day 10 - Real-Time Counter (RTC) Basics

Next, you will:

- Use the 32 kHz clock domain
- Generate accurate 1-second ticks
- Keep time while the CPU sleeps
- Build low-power periodic wake-up behavior

RTC is the timer you use when power matters and intervals are long.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>